HACKING BIG IRON WITH AI

Offensive Security on Mainframe Control Planes

Adam Toscher · Offensive Security / Red Team

IBM System/360 — 1964

SPEAKER

Adam Toscher

20+ years in offensive security, red teaming, and adversary simulation.

Started as an IBM intern porting Linux to mainframe — grounded in large-scale computing before transitioning to offensive security.

Senior roles spanning enterprise software, consulting, and government sectors.

Focus: penetration testing, red teaming, adversary emulation, and practical automation across private-sector and public infrastructure.

TALKS

Hushcon West '19
Cellular infrastructure security

Hushcon East '23
Hacker culture & attacker mindset

PRaSEC '23
Domain Admin attack paths

CypherCon '26
Hacking Big Iron with AI

Security Fest '26
Hacking Big Iron with AI

00

Before Big Iron Was Big Business

A brief history of the architecture everything still runs on — and how I ended up in a raised-floor lab writing REXX.

HISTORY

System/360: The Origin

The architecture every mainframe since 1964 still descends from

1964

IBM System/360 launched. Same ISA family still running today.

$5B

Development cost. More than the Manhattan Project adjusted. IBM bet the company.

East Fishkill, NY

IBM fabricated 360 chips here from 1964. Same Hudson Valley site later made chips for most consumer devices globally.

OS Lineage

OS/360 → MVS → MVS/ESA → OS/390 → z/OS. Backward compatible throughout.

HISTORY

The Promise That Shaped 60 Years of Security

Why the attack surface looks exactly like it does

IBM's 1964 Commitment

When the System/360 launched, IBM promised every program written for it would run on every future IBM mainframe. Forever. No exceptions. That promise was the entire sales pitch — buy into the platform once, your investment is protected permanently.

They Kept It

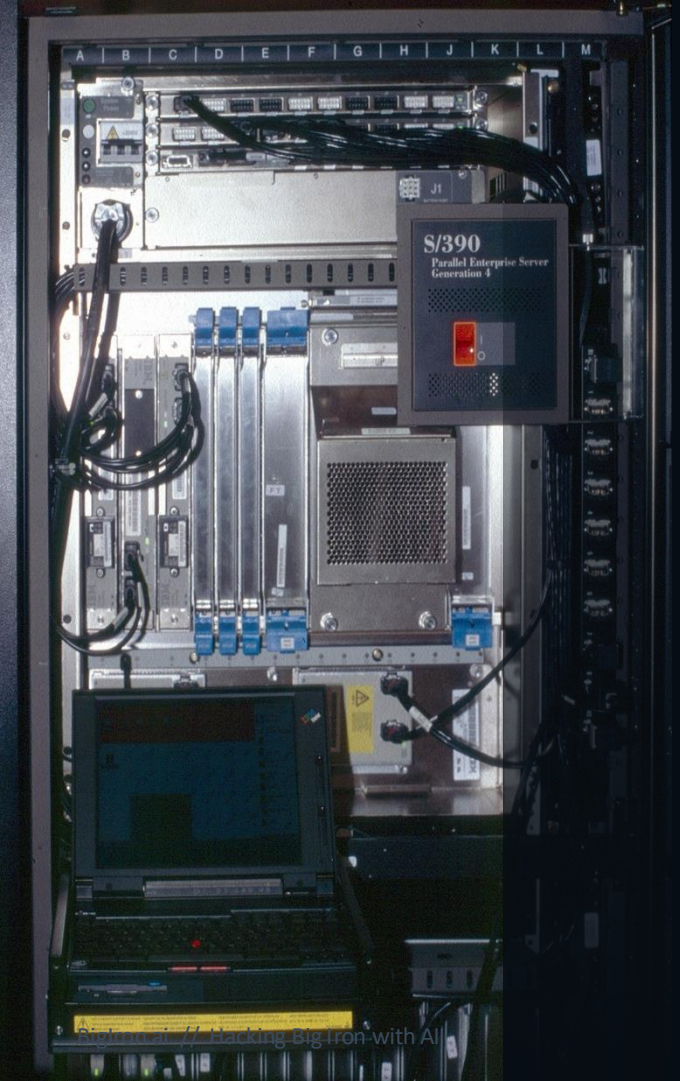
OS/360 code from 1967 runs on a z16 today, unmodified. Every subsystem built in the 1960s and 70s — RACF, JES, VTAM — is still there because removing it would break the promise. The backwards compatibility guarantee is contractual and architectural.

The Security Implication

Every design decision made before modern security thinking existed is still in production. JES deferred execution, VTAM session-based auth, RACF dataset-level controls — all of it designed decades before threat modeling was a concept. The attack surface is the price of the promise.

Nothing Gets Removed

Vendors patch. They deprecate. They remove dangerous features. IBM mainframes cannot. The compatibility guarantee means security teams inherit every architectural decision made since 1964. You don't fix it. You work around it.



HISTORY

From Room to Rack: MVS History

IBM S/390 Parallel Enterprise Server Gen 4 — a full mainframe in a rack

1970 System/370 — virtual memory added. MVS (Multiple Virtual Storage) becomes the dominant OS.

1990 System/390 (S/390) — ESA/390 architecture. MVS/ESA gains RACF, JES2/JES3 subsystems still in use today.

1995 OS/390 — unified product. Batch + CICS + DB2 + TCP/IP in one stack. Enterprise computing at scale.

2000 z/OS — 64-bit z/Architecture. Still fully backward compatible with 1964 S/360 application binaries.

Every security assumption you bring from Linux or Windows was built for a fundamentally different architectural model. RACF, JES, VTAM — none of it maps to anything in your mental model. That gap is the vulnerability.

PERSONAL HISTORY

Last IBM Internship. IBM HQ. The HMC.

Where I learned the ThinkPad wasn't just on the desk next to the mainframe. It was inside it.

The Hardware Management Console

IBM shipped ThinkPads physically bolted inside S/390 and zSeries cabinets as the HMC Support Element. Models 360, 365, 380, 600, T20, T30. Running OS/2. Hinged tray. Token ring. Permanently on.

IBM HQ — Last Internship

My last internship at IBM HQ is where I saw this in person. The ThinkPad wasn't a workstation sitting next to the mainframe. It was mounted inside the cabinet. It was the console. That reframe never left me.

The laptop I had been using to run z/OS was the same laptop IBM used to manage the machine that ran z/OS. Same ecosystem. Different chair.

HISTORY

REXX — The Scripting Layer That Matters

And why it shows up in attack paths

What REXX Is

REXX — Restructured Extended Executor — is the scripting language of z/OS. Created at IBM in 1979 by Mike Cowlishaw. It ships with every z/OS installation. No install required, no configuration. It is just there, available to any TSO user with the authority to run it.

What It Does

REXX scripts can call TSO commands, submit JCL, read and write datasets, invoke ISPF panels, and interact with every subsystem on the machine. Think of it as bash for z/OS — except tightly integrated into the execution model in ways that bash never is on Linux.

Why It Matters for Attacks

An attacker with TSO access and REXX authority can automate enumeration, stage data for exfiltration, and submit complex JCL sequences without using any tool that wasn't already on the box. No upload required. No binary dropped. REXX is native, trusted, and logged only if someone configured SMF to watch for it.

The ThinkPad Connection

IBM literally shipped ThinkPads bolted inside S/390 mainframes as Hardware Management Console Support Elements — ThinkPad models 360, 365, 380, 600, T20, T30. OS/2, Presentation Manager tools, token ring networking. The HMC ran on the same platform I was testing REXX on. That is not a coincidence — it is the same ecosystem.

01

The Real Problem

Most failures in mainframe security are conceptual, not technical. The issue is not access — it's misunderstanding how the system actually works.

CONTEXT

Why This Matters

30 B+

transactions/day

Banking, payment clearing, airline bookings — all Big Iron

73%

of Fortune 500 run z/OS

Most security teams assigned to them have zero mainframe training

0

EDR agents on z/OS

No process tree, no network telemetry, no SIEM by default

∞

API integrations adding exposure

Modern cloud layers connect to mainframe auth and execution

CONTEXT

The Compliance Fiction

How mainframes pass audits while having zero actual detection

RACF Policies Exist

Every mainframe environment has RACF configured. Access rules, group memberships, dataset protections. Auditors check the policy file and mark it compliant. What they don't check: whether anyone is reading the RACF violation logs. Spoiler — almost nobody is.

SMF Logging Is Enabled

SMF — Security Maintenance Facility — records are comprehensive. Every job submission, every dataset access, every RACF decision. It all gets written to SMF. What auditors don't ask: is anything parsing those records in real time? Is there a pipeline to a SIEM? Almost never.

The Gap

Having logs is not the same as having visibility. Having an access control policy is not the same as detecting violations. Mainframes pass PCI-DSS, SOX, and HIPAA audits regularly while having zero real-time detection capability. The checkbox gets checked. The attacker operates freely.

What This Means for Red Teams

You are not going to get caught by a SIEM alert. You are not going to trigger a behavioral detection. The risk is operational — submitting a job that crashes something — not detection-based. This changes the engagement posture entirely.

WRONG ASSUMPTIONS

Cloud Thinking Fails Here

"Find root, game over"	→	No root. Privilege is RACF profile-based — contextual, not binary
"Pivot through services"	→	No ports to pivot. Sessions route through VTAM. No shell, no netcat
"Shell access = control"	→	TSO is not bash. Execution is deferred through JES — submit and disconnect
"EDR will catch it"	→	Zero EDR agents on z/OS. SMF logs exist but real-time alerting is rare
"It's legacy — low priority"	→	Mainframes run 30B+ transactions/day. It's not legacy — it's load-bearing

MODERN Z/OS

The Real Attack Surface

What attackers are actually exploiting today — not your grandfather's mainframe

Unix System Services (USS)

z/OS ships a full POSIX layer. SSH, bash, Python, curl — all present. Philip Young's DEF CON 33 (SSH-nanigans) showed USS is now the primary pivot point.

CRITICAL

Network Job Entry (NJE)

Legacy job distribution protocol with minimal auth. Allows remote job submission between mainframes — effectively unauthenticated RCE if misconfigured.

CRITICAL

Buffer Overflows in z/OS Binaries

Discovered ~2020. C programs on z/OS are exploitable via save area overwrites and register-based techniques. Documented by Young & Rikansrud.

HIGH

Exposed TN3270 / FTP

Shodan finds hundreds of exposed mainframe services with cleartext auth, no rate limiting, weak passwords. Entry point for everything above.

HIGH

EXPOSURE

TN3270 on Shodan. Right Now.

port:23 "IBM" — cleartext auth, no rate limiting, internet-exposed

```
$ shodan search "port:23 IBM" --fields ip_str,port,org,hostnames

192.0.2.14        23    First National Bank         tn3270.fnb-
prod.internal
198.51.100.7      23    State Agency IT            mainframe1.agency.gov
203.0.113.88      23    MidWest Insurance Corp     mf-prod-01.mwi.com
192.0.2.201       23    Global Clearinghouse       clearing-mf.gcorp.net
198.51.100.33     23    Airline Reservations Co    res-mainframe.arc-
ops.com

... [UPDATE WITH REAL COUNT] results. Cleartext. No rate limit. No
lockout.

* IPs and hostnames illustrative — representative of real exposure patterns
```

0

rate limiting on port 23 by default

This is your initial access. Indexed. Available. Right now.

PRIOR ART

Standing on the Shoulders of Giants

This space has a history. Know it. Credit it.

Philip Young

aka Soldier of FORTRAN · Director, Mainframe Pentesting, NetSPI

- **2013** Black Hat — Mainframes: The Past Will Come Back to Haunt You
- **2014** DEF CON 22 — From root to SPECIAL: Pwning IBM Mainframes
- **2015** DEF CON 23 — Security Necromancy: Further Adventures
- **2025** DEF CON 33 — SSH-nanigans: Busting Open via Unix System Services

Others Who Paved It

Chad Rikansrud

Bigendian Smalls

Container breakout, USS exploitation, co-authored buffer overflow research

Ayoub (Ayoul3)

Wavestone

DEF CON 25 — privilege escalation paths on z/OS

Open Mainframe Project

Linux Foundation

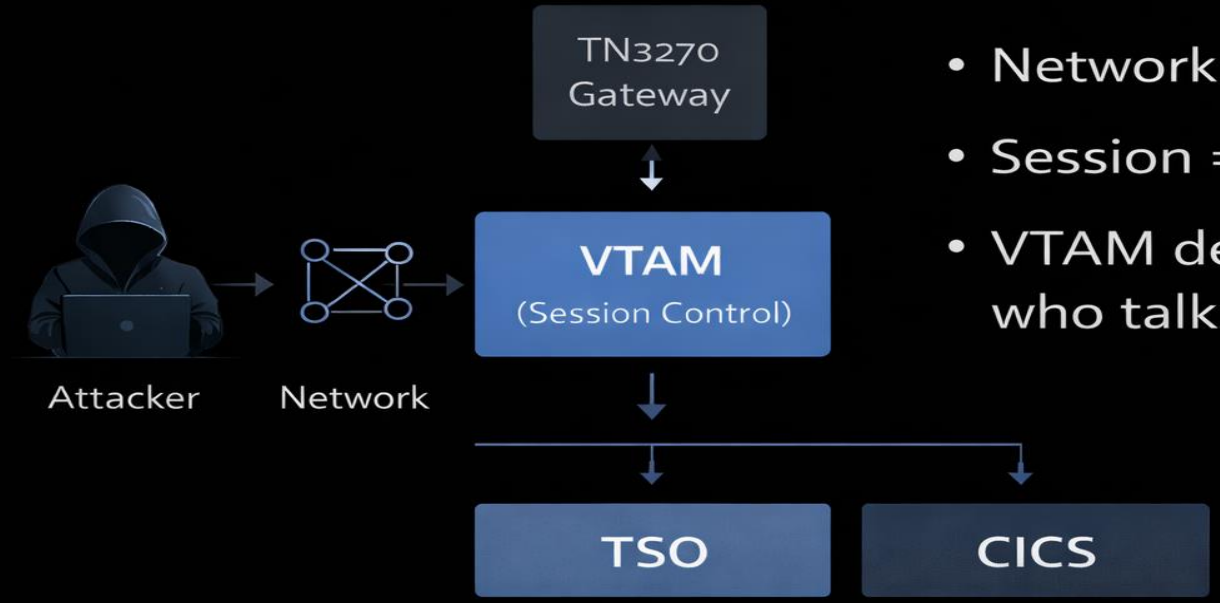
Education, tooling, Zowe framework

02

Mainframes Are Federated Systems

Authority is distributed across subsystems — not centralized in a kernel. Understanding which one you're in changes everything.

Exposure Is Session-Based (VTAM)



- Network ≠ Access
- Session = Exposure
- VTAM decides who talks to what

Security decisions are distributed across Subsystems, not centralized in the kernel.

ARCHITECTURE

The 5 Control Planes of MVS

IBM calls these subsystems. We call them control planes — the model that maps to modern security thinking.

VTAM	TSO/CICS	RACF	JES	MVS
Network / session fabric	Identity & interaction	Authorization engine	Deferred execution	The kernel
1	2	3	4	5

Attackers move between planes — not through ports or services.

ARCHITECTURE

Control Plane Transition — Worked Example

Attacker moves through planes — not ports, not exploits, not shells

PLANE 1
VTAM

TN3270 session from Shodan result. VTAM accepts. Attacker is in the session fabric.

No exploit. Just a connection.

PLANE 2
TSO

Credential from reuse or weak password. TSO login. RACF profile bound. Identity established.

No escalation yet.

PLANE 3
RACF

Enumerate group memberships, dataset access, SURROGAT grants. Find overpermissioned service account.

Authority path identified.

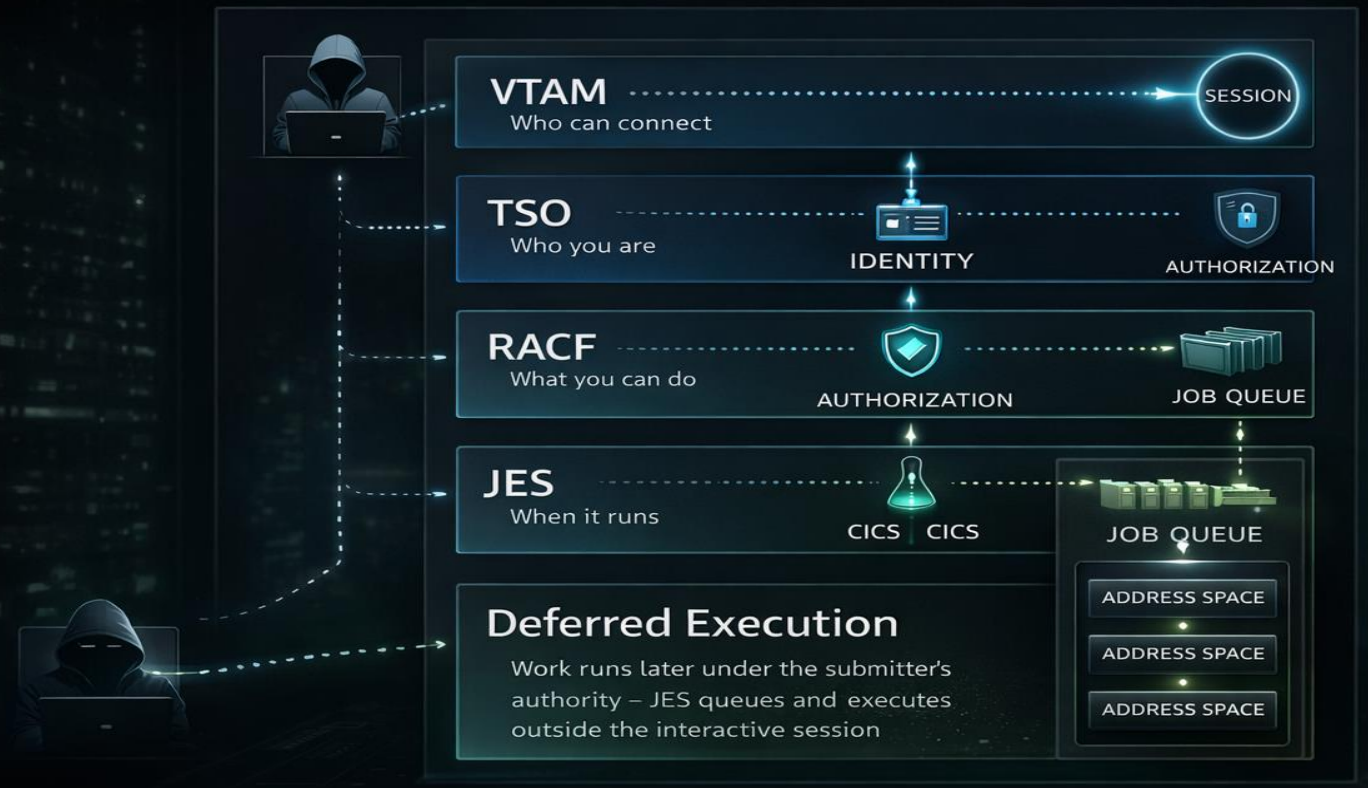
PLANE 4
JES

Submit JCL. Inherit authority from service account. Attacker logs off. Job fires later. No active session to detect.

Impact happens after you are gone.

ARCHITECTURE

Mainframe Control Planes & Deferred Execution



03

Attack Paths Follow Control Planes

Attackers don't move host-to-host. On mainframes, movement happens by changing control context — session to identity to execution.

ENTRY

Realistic Entry Path

Initial access rarely requires exploitation

TN3270 Exposed → User Enum → Cred Access → TSO Login → Submit JCL → Priv Exec

Entry Vectors

- Exposed TN3270 (Shodan-findable)
- Shared or reused TSO credentials
- Vendor or service accounts
- Web / middleware trust chains

Execution Sequence

- VTAM → session establishment
- TSO → identity binding + RACF context
- JES → deferred job submission
- Impact after logoff, under system context

Key Insight

- Access now, impact later
- No exploitation required
- Authority inherited, not escalated
- Attacker may be long gone before job fires

ATTACK PATH

Privilege Escalation: The Trust Ladder

5 — SYSTEM AUTHORITY

APF library access → authorized execution → supervisor state

4 — TRUSTED EXECUTION

Started tasks, system utilities, automation workflows

3 — BATCH AUTHORITY

JES job influence — parameters, input datasets, PROCs

2 — IDENTITY CONTEXT

SURROGAT abuse, shared / service accounts

1 — USER ACCESS

Normal TSO or application access

Influence → Inherit → Control

ATTACK PATH

Real RACF Misconfigurations — What You Actually Find

Anonymized from real enterprise environments

SURROGAT Overprovisioning

Service accounts granted SURROGAT authority over privileged users — allowing them to act as those users. Legitimate for automation. Dangerous when the service account's credentials are shared, stale, or accessible to non-privileged staff. Common in orgs where integrations were built fast and never reviewed.

The SPECIAL Attribute

RACF's SPECIAL attribute grants the ability to manage all RACF profiles system-wide. It should exist on maybe two accounts. Typical enterprise finding: 15-30 accounts with SPECIAL, including service accounts for monitoring tools, batch jobs, and middleware that haven't been reviewed in years.

Overpermissive Dataset Rules

Dataset profiles using wildcards like SYS1.** with READ access granted to large groups. SYS1 datasets include APF libraries, authorized program lists, and system catalogs. READ on the wrong SYS1 dataset gives an attacker a map to every privileged execution path on the machine.

Stale Vendor Accounts

TSO accounts created for IBM, software vendors, or consulting engagements. Never disabled after the engagement ended. Default or never-rotated passwords. Group memberships from when they needed access to something five years ago. Still active. Still connected. Still on Shodan.

CASE STUDY

STEPLIB Hijack / JCL Injection

CONCEPT MAPPING

STEPLIB override	=	LD_PRELOAD / PATH hijack
Deferred execution	=	Cron job w/ user script
Identity inheritance	=	Setuid binary

WHY IT'S DANGEROUS

- No vulnerability exploited
- No memory corruption or shellcode
- RACF controls datasets — not library search order
- No alerts by default — SMF records job but nobody watches
- Job runs even after logoff

The mainframe equivalent of a supply chain attack.

DEFENDER VIEW

Operational Reality: Why You Can't See It

What's Missing

- No EDR agents — no process tree, no behavioral telemetry
- No real-time session monitoring — SMF is batch, not streaming
- Execution outside interactive sessions via JES batch
- Logging fragmented across VTAM, TSO, RACF, JES, CICS
- Long-lived address spaces blur behavioral context

What This Means

- Attacker submits job, logs off — no active session to detect
- SMF records exist but require specialized tooling to parse
- SIEM integration is non-default and rarely configured
- Security visibility depends entirely on manual configuration
- Modern SOC tooling is essentially blind here

Splunk Enterprise | Search & Reporting | index=mainframe sourcetype=zos | Last 24h | 0 results

Nothing.

The attacker has been on the system for 6 hours. The SIEM has nothing to show you.

04

BigIron.ai

The methodology made operational. A reference implementation of the control-plane reasoning model — 100% local, open-source.

LIVE DEMO

AI-POWERED MAINFRAME OPERATIONS

MVS 3.8j Turnkey system. This is a free, open source mainframe OS running on Hercules emulator.

Home
AI Chat
Models
Terminal
Present
Labs
Scanner
Test
RAG
Tutor
Walkthru
Graph
Arch
Docs

TN3270 Terminal – IBM Mainframe

Chat Capture Host: localhost:327 DISCONNECT Connected: localhost:3270

```
Terminal   CUU0C0                                          Date  09.02.26
                                                           Time  01:48:35

                         TTTTTTTTTTTT   KKKK  KKKKK    555555555555
                         TT   TT   TT    KK    KK      55
                         TT   TT   TT    KK   KK       55       Update 4
                              TT         KK  KK        55
        |l      _,,,---,,_   TT         KK KK         55
 ZZZzz /,'.-''    -.  ;-;;,  TT         KKKK          55555555555
     |,4-  ) )-,_. ,( (  ''-' TT         KKKKK                    55
    '---''(_/--'  `-')_)     TT         KK  KK                   55
                              TT         KK   KK                 55
      The MVS 3.8j            TT         KK    KK                55
    Tur(n)key System          TT         KK     KK               55
                            TTTTTT      KKKK     KKK   55555555555

     TK3  created by Volker Bandke      volker@bandke.org
     TK4- update by Juergen Winkelmann  juergen.winkelmann@pebble-beach.ch
     TK5  update by Rob Prins           prin0096@gmail.com
          see SYS2.JCLLIB(CREDITS) for complete credits
                        MVS 3.8j Level 8505

Logon ===>
                                                        RUNNING  TK5
```

Enter Clear Tab Reset F1 F2 F3 F4 F5 F6 F7 F8 F9 F10 F11 F12

OS/2 Warp – Mainframe AI Assistant Connected – localhost:3270

WarpCenter TN3270 Terminal 20:55

LLM: **Connected** MODEL: **llama3.1:8b** TN3270: **localhost:3270** AGENT: **General Assistant** RAG: **3 sources**

SYSTEM

IBM Mainframe AI Assistant Ready

I can help you with:
- ABEND codes and error analysis
- JCL generation and debugging
- COBOL, REXX, and Assembler
- TSO/ISPF and CICS navigation
- Live 3270 terminal control

Connected RAG Sources: Mainframe Documentation, ABEND Codes, JCL Templates

Type `/connect localhost:3270` to connect to MVS, or click the **Terminal** tab.

> CONNECTION STATUS

Ollama	ONLINE
Model	llama3.1:8b
TN3270	ONLINE
Host	localhost:3270

> ACTIVE AGENT

Name	General
Mode	Conversational

> RAG SOURCES

Mainframe Docs	ACTIVE
ABEND Codes	ACTIVE
JCL Templates	ACTIVE

LABS

Session Stack Walkthrough 4 steps
Walk the VTAM -> TSO -> ISPF layers and learn where trust boundaries shift.
Beginner Not started

Batch Execution Basics 3 steps
Submit a simple job and map the JCL -> JES -> program execution chain.
Beginner Not started

RACF Authorization Model 4 steps
Understand how RACF enforces access control on datasets, programs, and resources.
Beginner Not started

Dataset Enumeration 4 steps
Discover and catalog datasets to map the system's data landscape.
Intermediate Not started

TSO User Enumeration 4 steps
Discover valid TSO userids through logon response analysis.
Intermediate Not started

JCL Security Analysis 4 steps
Analyze JCL for security implications and privilege escalation paths.
Intermediate Not started

Hidden Field Detection 4 steps
Identify non-display fields that may contain sensitive data or bypass controls.
Advanced Not started

TN3270 TERMINAL

localhost:3270 DISCONNECT

```
Hercules Version  : 4.9.1.0-SDL
                                                        Time  14:22:10

                    TTTTTTTTTTTT   KKKK  KKKKK    555555555555
                    TT    TT   TT   KK    KK     55
                    TT    TT   TT   KK   KK      55        Update 4
                       TT           KK  KK       55
        |l      _,,,---,,_          TT          KK KK         55
ZZZzz /,'.-'``    -.  ;-;;,         TT          KKKK          55555555555
     |,4-  ) )-,_. ,( (  ''-'       TT          KKKKK                   55
    '---''(_/--'  `-')_)            TT          KK  KK                  55
                                    TT          KK   KK                 55
      The MVS 3.8j                  TT          KK    KK                55
    Tur(n)key System                TT          KK     KK               55
                                  TTTTTT      KKKK     KKK   55555555555

     TK3  created by Volker Bandke      volker@bandke.org
     TK4- update by Juergen Winkelmann  juergen.winkelmann@pebble-beach.ch
     TK5  update by Rob Prins           prin0096@gmail.com
          see SYS2.JCLLIB(CREDITS) for complete credits
                     MVS 3.8j Level 8505

Logon ===>
                                                        RUNNING  TK5
```

ENTER CLEAR TAB PF1 PF3 PF7 PF8

TUTOR

Welcome to Red Team Tutor

Learn mainframe security through guided exploration.

RED TEAM FOCUS
- Map the session stack (VTAM → TSO → ISPF)
- Spot access controls (RACF, dataset profiles)
- Find weak links (permissive datasets, defaults)
- Capture evidence (screens, JES output, logs)

STARTER OBJECTIVES
- Connect to TK5 and reach the TSO logon
- List jobs in JES spool and read output
- Inspect dataset security metadata
- Ask the tutor to explain a screen or error

ENGAGEMENT CHECKLIST
- Scope: TK5 lab only, no external targets
- Evidence: note steps, panels, timestamps
- Minimize: change nothing unless instructed
- Exit: document findings and clean up

COMMON PITFALLS
- PF keys vary; try PF3 to exit, PF7/PF8 to page
- Uppercase matters in TSO and dataset names
- JES output appears after job completes
- ISPF panels may hide commands in line 1

FIRST 5 COMMANDS
- TSO: `LOGON USET` then password `PASS`

What am I looking at? | What do I type? | Explain error | Engagement checklist | Common pitfalls | First 5 commands

Ask the tutor... SEND

CAPABILITIES

BigIron.ai — Full Capability Set

AI TN3270 Terminal

Click-to-analyze — LLM explains context, corrects wrong assumptions

13 Autonomous Walkthroughs

Hands-free scripted demos across all 6 control planes

BloodHound-style Graph

Trust relationships and indirect execution paths visualized

F1–F5 Findings Engine

Control-plane framework with JSON/MD/HTML report gen

RAG Knowledge Base

Local IBM Redbook + ABEND table ingestion for context-aware answers

TN3270 Network Scanner

Mainframe discovery — the nmap for Big Iron

PR/SM HMC Simulator

LLM-driven LPAR exploration — IBM z16 hardware console emulation

100% Local / Offline

Ollama inference — no cloud, no data exfiltration, no API keys

Why 100% Offline Is Not Optional

The organizations that run mainframes cannot use cloud AI

PCI-DSS Scope

Mainframes processing payment card data are in PCI-DSS scope. Sending cardholder data, system outputs, or assessment artifacts to a cloud AI provider during an engagement creates a scope violation. No cloud AI. Full stop. The offline model is the only compliant option.

Air-Gapped Environments

Government agencies, defense contractors, and utilities running z/OS frequently operate in air-gapped or highly restricted network environments. No outbound internet. No API calls to external services. If the tool needs a cloud endpoint it doesn't run. BigIron.ai runs on Ollama with no network dependency.

SWIFT CSP Requirements

Financial institutions connected to SWIFT operate under the Customer Security Programme. Mandatory controls restrict what data can leave the environment and how. An AI-assisted assessment tool that phones home to OpenAI or Anthropic during a SWIFT-connected mainframe engagement fails these controls immediately.

Data Residency

European financial institutions under DORA and GDPR, healthcare organizations under HIPAA, and regulated utilities under NERC CIP all face strict data residency requirements. Mainframe assessment output — RACF profiles, SMF data, job logs — cannot leave the jurisdiction. The inference engine has to be local.

THE SHIFT

Why AI Changes the Equation

BEFORE

- ✗ Mainframe expertise took years to build
- ✗ Assessment required a Philip Young — rare, expensive, hard to find
- ✗ TN3270 screens are opaque without deep domain knowledge
- ✗ One wrong command — job submitted, no undo
- ✗ Most red teams skip it entirely

→

WITH BIGIRON.AI

- ✓ LLM explains every screen in plain English — in real time
- ✓ Methodology encoded — Q1–Q4 framework runs automatically
- ✓ Wrong assumptions flagged before you act on them
- ✓ Read-only by default — guided exploration, not blind execution
- ✓ Any red teamer can begin a structured assessment on day one

AI doesn't replace Philip Young. It raises the floor so more teams can enter the conversation.

// THE ONE LINE THAT MATTERS

Mainframes are not secured by services and ports.

They are secured by control over execution paths and dataset authority.

Stop importing cloud assumptions. Accurate interpretation of architecture is more valuable than exploit hunting in these environments.